SQL Solution Manual (Part 2)

Levels 5 through 8 (Questions 111–200) | CTEs, Window Functions, Advanced Patterns

L5 — Subqueries, EXISTS & Set Operations (Questions 111–140)

> **Focus:** Correlated subqueries, EXISTS, NOT EXISTS, UNION, INTERSECT, EXCEPT.

Q111. Find employees earning more than the average salary.

```
SELECT emp_name, salary FROM employees WHERE salary > (SELECT AVG(salary) FROM
employees);
```

SAMPLE OUTPUT:

emp_name	salary
Sarah Connor	120000
Diana	75000
Ian	150000

Explanation: A simple scalar subquery. The inner query evaluates once to a single value, and the outer query filters against it.

Q112. Find employees earning the maximum salary.

```
SELECT emp_name, salary FROM employees WHERE salary = (SELECT MAX(salary) FROM
employees);
```

SAMPLE OUTPUT:

emp_name	salary
Ian	150000

Explanation: Standard pattern for finding the record holding a specific aggregate extreme.

Q113. Find employees earning the minimum salary.

```
SELECT emp_name, salary FROM employees WHERE salary = (SELECT MIN(salary) FROM
employees);
```

SAMPLE OUTPUT:

emp_name	salary
Dan	35000

Explanation: Inverse of the maximum scalar subquery pattern.

Q114. Find employees earning more than the average salary of department 10.

```
SELECT emp_name, salary FROM employees WHERE salary > (SELECT AVG(salary) FROM
employees WHERE dept_id = 10);
```

SAMPLE OUTPUT:

emp_name	salary
Sarah Connor	120000
Diana	75000
Ian	150000

Explanation: The scalar subquery can have its own filtering independent of the outer query.

Q115. Find employees whose salary is greater than their department's average salary.

```
SELECT e.emp_name, e.salary, e.dept_id FROM employees e WHERE e.salary > (SELECT
AVG(salary) FROM employees sub WHERE sub.dept_id = e.dept_id);
```

SAMPLE OUTPUT:

emp_name	salary	dept_id
Sarah Connor	120000	10
Diana	75000	30
Evan	60000	20

Explanation: A Correlated Subquery. The inner query references 'e.dept_id' from the outer query, meaning it must run once for *every* row in the outer table. (Note: Window functions are generally better for this now).

Q116. Find departments whose average salary is greater than the company average.

```
SELECT dept_id, AVG(salary) FROM employees GROUP BY dept_id HAVING AVG(salary) >
(SELECT AVG(salary) FROM employees);
```

SAMPLE OUTPUT:

dept_id	AVG(salary)
40	150000.0

Explanation: Subquery inside a HAVING clause. A very common interview pattern.

Q117. Find employees working in the department with the highest average salary.

```
SELECT emp_name FROM employees WHERE dept_id = (SELECT dept_id FROM employees GROUP BY
dept_id ORDER BY AVG(salary) DESC LIMIT 1);
```

SAMPLE OUTPUT:

emp_name
Ian

Explanation: Subquery returning a single ID by ordering and limiting.

Q118. Find the second-highest salary.

```
SELECT MAX(salary) FROM employees WHERE salary < (SELECT MAX(salary) FROM employees);
-- OR USING LIMIT/OFFSET:
-- SELECT DISTINCT salary FROM employees ORDER BY salary DESC LIMIT 1 OFFSET 1;
```

SAMPLE OUTPUT:

MAX(salary)
120000

Explanation: The subquery approach is ANSI standard. The LIMIT/OFFSET approach is MySQL/PostgreSQL specific but very common.

Q119. Find the third-highest salary.

```
SELECT DISTINCT salary FROM employees ORDER BY salary DESC LIMIT 1 OFFSET 2;
```

SAMPLE OUTPUT:

salary
75000

Explanation: OFFSET skips the first N records. To get the 3rd, we skip 2 (OFFSET 2) and take 1 (LIMIT 1).

Q120. Find the 4th-highest distinct salary (Nth highest pattern).

```
SELECT DISTINCT salary FROM employees ORDER BY salary DESC LIMIT 1 OFFSET 3;
```

SAMPLE OUTPUT:

salary
60000

Explanation: Formula for Nth highest: LIMIT 1 OFFSET (N-1).

Q121. Find employees who earn the second-highest salary.

```
SELECT emp_name, salary FROM employees WHERE salary = (SELECT DISTINCT salary FROM
employees ORDER BY salary DESC LIMIT 1 OFFSET 1);
```

SAMPLE OUTPUT:

emp_name	salary
Sarah Connor	120000

Explanation: Combine the 'find Nth salary' subquery with an outer query to fetch the actual employee record.

Q122. Find departments containing at least one employee earning more than 100,000.

```
SELECT dept_name FROM departments d WHERE EXISTS (SELECT 1 FROM employees e WHERE
e.dept_id = d.dept_id AND e.salary > 100000);
```

SAMPLE OUTPUT:

dept_name
IT
Sales

Explanation: EXISTS stops scanning as soon as it finds the first match, making it highly efficient for 'at least one' questions.

Q123. Find departments where every employee earns more than 40,000.

```
SELECT dept_name FROM departments d WHERE NOT EXISTS (SELECT 1 FROM employees e WHERE
e.dept_id = d.dept_id AND e.salary <= 40000);
```

SAMPLE OUTPUT:

dept_name
HR
Sales
Operations
EmptyDept

Explanation: Double negative logic: Find departments where there DOES NOT exist an employee earning <= 40000.

Q124. Find customers who have placed at least one order using EXISTS.

```
SELECT customer_name FROM customers c WHERE EXISTS (SELECT 1 FROM orders o WHERE
o.customer_id = c.customer_id);
```

SAMPLE OUTPUT:

customer_name
Whale Customer
Regular Joe

Explanation: Often performs better than IN or JOIN for simple existence checks, especially if the orders table is massive.

Q125. Find customers who have never placed an order using NOT EXISTS.

```
SELECT customer_name FROM customers c WHERE NOT EXISTS (SELECT 1 FROM orders o WHERE
o.customer_id = c.customer_id);
```

SAMPLE OUTPUT:

customer_name
Window Shopper

Explanation: The subquery alternative to the Anti-Join pattern.

Q126. Find products that have never appeared in an order.

```
SELECT product_name FROM products p WHERE NOT EXISTS (SELECT 1 FROM order_items oi
WHERE oi.product_id = p.product_id);
```

SAMPLE OUTPUT:

product_name
T-Shirt
Unsold Item

Explanation: Inventory check using NOT EXISTS.

Q127. Find employees who work in departments located in Hyderabad.

```
SELECT emp_name FROM employees WHERE dept_id IN (SELECT dept_id FROM departments WHERE location = 'Hyderabad');
```

SAMPLE OUTPUT:

emp_name
Sarah Connor
Alice Smith
Alan
John

Explanation: Subquery using IN. It evaluates the inner query to a list of IDs, then filters the outer query.

Q128. Find employees who earn more than every employee in department 20.

```
SELECT emp_name, salary FROM employees WHERE salary > ALL (SELECT salary FROM employees WHERE dept_id = 20);
```

SAMPLE OUTPUT:

Query requires dialect-specific features. Valid in MySQL. SQLite Error: near "ALL": syntax error

Explanation: The ALL operator ensures the condition is true for *every* value returned by the subquery.

Q129. Find employees who earn more than at least one employee in department 20.

```
SELECT emp_name, salary FROM employees WHERE salary > ANY (SELECT salary FROM employees WHERE dept_id = 20);
```

SAMPLE OUTPUT:

Query requires dialect-specific features. Valid in MySQL. SQLite Error: near "SELECT": syntax error

Explanation: The ANY operator ensures the condition is true for *at least one* value returned by the subquery.

Q130. Find departments whose employee count is greater than the average department employee count.

```
SELECT dept_id, COUNT(*) FROM employees GROUP BY dept_id HAVING COUNT(*) > (SELECT COUNT(*)/COUNT(DISTINCT dept_id) FROM employees);
```

SAMPLE OUTPUT:

dept_id	COUNT(*)
10	4

Explanation: A complex HAVING subquery calculating the global average headcount.

Q131. Find the employee with the highest salary in each department using a correlated subquery.

```
SELECT e1.emp_name, e1.salary, e1.dept_id FROM employees e1 WHERE e1.salary = (SELECT
MAX(salary) FROM employees e2 WHERE e1.dept_id = e2.dept_id);
```

SAMPLE OUTPUT:

emp_name	salary	dept_id
Sarah Connor	120000	10
Diana	75000	30
Evan	60000	20
Ian	150000	40

Explanation: A classic interview pattern before Window Functions became standard. It matches the outer row's salary to the max salary calculated for that specific department.

Q132. Find employees whose salary is greater than their manager's salary using a correlated subquery.

```
SELECT e.emp_name FROM employees e WHERE e.salary > (SELECT salary FROM employees m
WHERE m.emp_id = e.manager_id);
```

SAMPLE OUTPUT:

emp_name
Evan
Ian

Explanation: An alternative to the Self-Join pattern we used in L4.

Q133. Find customers whose spending is greater than the average customer spending.

```
SELECT customer_id, SUM(total_amount) FROM orders GROUP BY customer_id HAVING
SUM(total_amount) > (SELECT SUM(total_amount)/COUNT(DISTINCT customer_id) FROM orders);
```

SAMPLE OUTPUT:

customer_id	SUM(total_amount)
1	6150

Explanation: Similar to Q130, dealing with aggregated averages.

Q134. Find products whose revenue is greater than the average product revenue.

```
SELECT product_id, SUM(quantity * unit_price) FROM order_items GROUP BY product_id
HAVING SUM(quantity * unit_price) > (SELECT SUM(quantity * unit_price)/COUNT(DISTINCT
product_id) FROM order_items);
```

SAMPLE OUTPUT:

product_id	SUM(quantity * unit_price)
101	5000

Explanation: Complex revenue comparison using subqueries.

Q135. Combine employee names from two employee tables using UNION.

```
SELECT emp_name FROM employees UNION SELECT emp_name FROM archived_employees;
```

SAMPLE OUTPUT:

emp_name
Alan
Alice Smith
Brian
Dan
Diana
Evan
Ian
John
Old Bob
Orphan Employee
Sarah Connor

Explanation: UNION combines result sets vertically and automatically removes duplicates. Both SELECTs must have the same number of columns of similar types.

Q136. Combine two employee datasets while preserving duplicates using UNION ALL.

```
SELECT emp_name FROM employees UNION ALL SELECT emp_name FROM archived_employees;
```

SAMPLE OUTPUT:

emp_name
Sarah Connor
Alice Smith
Brian
Alan
Dan
Diana
Evan
Orphan Employee
John
Ian
Old Bob

Explanation: UNION ALL does not deduplicate, making it much faster. Always use UNION ALL unless you explicitly need distinct results.

Q137. Find records appearing in both datasets using INTERSECT.

```
SELECT emp_id FROM employees INTERSECT SELECT emp_id FROM archived_employees;
```

SAMPLE OUTPUT:

OUTPUT: (0 ROWS RETURNED MATCHING CRITERIA)

Explanation: INTERSECT returns only rows that exist in *both* queries. (Note: Supported in MySQL 8.0.31+; in older versions, use INNER JOIN).

Q138. Find records appearing in the first dataset but not the second using EXCEPT.

```
SELECT emp_id FROM employees EXCEPT SELECT emp_id FROM archived_employees;
```

SAMPLE OUTPUT:

emp_id
1
2
3
4
5
6
7
8
9
10

Explanation: EXCEPT (or MINUS in Oracle) returns rows from the first query that do *not* exist in the second query.

Q139. Find customers who purchased both Product 101 and Product 102.

```
SELECT customer_id FROM orders o JOIN order_items oi ON o.order_id = oi.order_id WHERE
oi.product_id = 101 INTERSECT SELECT customer_id FROM orders o JOIN order_items oi ON
o.order_id = oi.order_id WHERE oi.product_id = 102;
```

SAMPLE OUTPUT:

customer_id
1

Explanation: Relational Division problem solved gracefully with INTERSECT.

Q140. Find customers who purchased Product 101 but never purchased Product 102.

```
SELECT customer_id FROM orders o JOIN order_items oi ON o.order_id = oi.order_id WHERE
oi.product_id = 101 EXCEPT SELECT customer_id FROM orders o JOIN order_items oi ON
o.order_id = oi.order_id WHERE oi.product_id = 102;
```

SAMPLE OUTPUT:

OUTPUT: (0 ROWS RETURNED MATCHING CRITERIA)

Explanation: Applying EXCEPT for exclusion logic based on items.

L6 — CTEs + Window Functions (Questions 141–165)

> **Focus:** WITH clause, ROW_NUMBER(), RANK(), DENSE_RANK(), LAG(), LEAD().

Q141. Create a CTE containing employees earning more than 60,000.

```
WITH HighEarners AS (SELECT * FROM employees WHERE salary > 60000) SELECT emp_name FROM HighEarners;
```

SAMPLE OUTPUT:

emp_name
Sarah Connor
Diana
Ian

Explanation: A CTE (Common Table Expression) creates a temporary, named result set that exists only for the duration of the query. Great for readability.

Q142. Use a CTE to calculate average salary by department.

```
WITH DeptAvg AS (SELECT dept_id, AVG(salary) as avg_sal FROM employees GROUP BY dept_id) SELECT * FROM DeptAvg;
```

SAMPLE OUTPUT:

dept_id	avg_sal
NULL	45000.0
10	67500.0
20	55000.0
30	55000.0

Explanation: Isolating the aggregation step makes outer queries much cleaner.

Q143. Use a CTE to find departments whose average salary exceeds the company average.

```
WITH DeptAvg AS (SELECT dept_id, AVG(salary) as avg_sal FROM employees GROUP BY dept_id), CompAvg AS (SELECT AVG(salary) as global_avg FROM employees) SELECT d.dept_id FROM DeptAvg d JOIN CompAvg c ON d.avg_sal > c.global_avg;
```

SAMPLE OUTPUT:

dept_id
40

Explanation: You can define multiple CTEs separated by commas. This avoids deeply nested subqueries.

Q144. Use multiple CTEs to calculate customer spending and compare it with average spending.

```
WITH CustSpend AS (SELECT customer_id, SUM(total_amount) as spend FROM orders GROUP BY customer_id), AvgSpend AS (SELECT AVG(spend) as global_avg FROM CustSpend) SELECT c.customer_id, c.spend FROM CustSpend c JOIN AvgSpend a ON c.spend > a.global_avg;
```

SAMPLE OUTPUT:

customer_id	spend
1	6150

Explanation: Notice how the second CTE (AvgSpend) references the first CTE (CustSpend).

Q145. Use a CTE to calculate total revenue by product and return the top 5.

```
WITH ProdRev AS (SELECT product_id, SUM(quantity * unit_price) as rev FROM order_items GROUP BY product_id) SELECT * FROM ProdRev ORDER BY rev DESC LIMIT 5;
```

SAMPLE OUTPUT:

product_id	rev
101	5000
102	1500
103	150

Explanation: Simplifies top-N queries.

Q146. Assign a row number to employees ordered by salary.

```
SELECT emp_name, salary, ROW_NUMBER() OVER (ORDER BY salary DESC) as rn FROM employees;
```

SAMPLE OUTPUT:

emp_name	salary	rn
Ian	150000	1
Sarah Connor	120000	2
Diana	75000	3
Alice Smith	60000	4

Explanation: ROW_NUMBER() assigns a unique, sequential integer starting at 1 based on the ORDER BY clause within the OVER() window.

Q147. Assign row numbers separately within each department.

```
SELECT emp_name, dept_id, salary, ROW_NUMBER() OVER (PARTITION BY dept_id ORDER BY salary DESC) as dept_rn FROM employees;
```

SAMPLE OUTPUT:

emp_name	dept_id	salary	dept_rn
Orphan Employee	NULL	45000	1
Sarah Connor	10	120000	1
Alice Smith	10	60000	2
John	10	50000	3

Explanation: PARTITION BY resets the row number counter for each unique value of dept_id.

Q148. Rank employees by salary.

```
SELECT emp_name, salary, RANK() OVER (ORDER BY salary DESC) as rank_val FROM employees;
```

SAMPLE OUTPUT:

emp_name	salary	rank_val
Ian	150000	1
Sarah Connor	120000	2
Diana	75000	3
Alice Smith	60000	4

Explanation: RANK() behaves like ROW_NUMBER(), but assigns the same rank to ties. It leaves gaps (e.g., 1, 2, 2, 4).

Q149. Compare RANK() and DENSE_RANK() for duplicate salaries.

```
SELECT emp_name, salary, RANK() OVER (ORDER BY salary DESC) as rnk, DENSE_RANK() OVER (ORDER BY salary DESC) as drnk FROM employees;
```

SAMPLE OUTPUT:

emp_name	salary	rnk	drnk
Ian	150000	1	1
Sarah Connor	120000	2	2
Diana	75000	3	3
Alice Smith	60000	4	4

Explanation: DENSE_RANK() does not leave gaps after ties (e.g., 1, 2, 2, 3). Know the difference for interviews!

Q150. Find the highest-paid employee in each department using ROW_NUMBER().

```
WITH RankedEmps AS (SELECT emp_name, dept_id, salary, ROW_NUMBER() OVER (PARTITION BY dept_id ORDER BY salary DESC) as rn FROM employees) SELECT emp_name, dept_id, salary FROM RankedEmps WHERE rn = 1;
```

SAMPLE OUTPUT:

emp_name	dept_id	salary
Orphan Employee	NULL	45000
Sarah Connor	10	120000
Evan	20	60000
Diana	30	75000

Explanation: The ultimate interview pattern. Window functions cannot be put in the WHERE clause, so you MUST wrap them in a CTE (or subquery) first, then filter where rn=1.

Q151. Find the top three highest-paid employees in each department.

```
WITH RankedEmps AS (SELECT emp_name, dept_id, salary, DENSE_RANK() OVER (PARTITION BY dept_id ORDER BY salary DESC) as rnk FROM employees) SELECT emp_name, dept_id, salary FROM RankedEmps WHERE rnk <= 3;
```

SAMPLE OUTPUT:

emp_name	dept_id	salary
Orphan Employee	NULL	45000
Sarah Connor	10	120000
Alice Smith	10	60000
John	10	50000

Explanation: DENSE_RANK is safer for top-N queries if ties exist, guaranteeing you actually get the top 3 salary tiers.

Q152. Find the second-highest-paid employee in each department.

```
WITH RankedEmps AS (SELECT emp_name, dept_id, salary, DENSE_RANK() OVER (PARTITION BY dept_id ORDER BY salary DESC) as rnk FROM employees) SELECT emp_name, dept_id, salary FROM RankedEmps WHERE rnk = 2;
```

SAMPLE OUTPUT:

emp_name	dept_id	salary
Alice Smith	10	60000
Brian	20	50000
Dan	30	35000

Explanation: Easily fetches the Nth highest per partition without complex correlated subqueries.

Q153. Find the third-highest-paid employee in each department.

```
WITH RankedEmps AS (SELECT emp_name, dept_id, salary, DENSE_RANK() OVER (PARTITION BY dept_id ORDER BY salary DESC) as rnk FROM employees) SELECT emp_name, dept_id, salary FROM RankedEmps WHERE rnk = 3;
```

SAMPLE OUTPUT:

emp_name	dept_id	salary
John	10	50000

Explanation: Same logic, just changing the filter threshold.

Q154. Find employees whose salary is above their department's average using a window function.

```
WITH DeptStats AS (SELECT emp_name, dept_id, salary, AVG(salary) OVER (PARTITION BY dept_id) as dept_avg FROM employees) SELECT emp_name FROM DeptStats WHERE salary > dept_avg;
```

SAMPLE OUTPUT:

emp_name
Sarah Connor
Evan
Diana

Explanation: Using aggregate functions (like AVG) as a Window Function attaches the group's result to every individual row without collapsing the rows.

Q155. Calculate the running total of salaries ordered by joining date.

```
SELECT emp_name, hire_date, salary, SUM(salary) OVER (ORDER BY hire_date ASC) as running_total FROM employees;
```

SAMPLE OUTPUT:

emp_name	hire_date	salary	running_total
Sarah Connor	2020-01-15	120000	120000
Ian	2021-06-15	150000	270000
Dan	2022-11-01	35000	305000
Alice Smith	2023-02-10	60000	365000

Explanation: If you use an ORDER BY without a PARTITION BY in a SUM() window function, it defaults to calculating a cumulative running total.

Q156. Calculate cumulative revenue by order date.

```
SELECT order_date, total_amount, SUM(total_amount) OVER (ORDER BY order_date ASC) as cum_revenue FROM orders;
```

SAMPLE OUTPUT:

order_date	total_amount	cum_revenue
2023-01-10	1500	1500
2023-02-15	550	2050
2023-03-20	1000	3050
2023-04-25	1050	4100

Explanation: Standard pattern for calculating cumulative financial metrics over time.

Q157. Calculate the average salary of the current employee's department using a window function.

```
SELECT emp_name, dept_id, salary, AVG(salary) OVER (PARTITION BY dept_id) as dept_avg FROM employees;
```

SAMPLE OUTPUT:

emp_name	dept_id	salary	dept_avg
Orphan Employee	NULL	45000	45000.0
Sarah Connor	10	120000	67500.0
Alice Smith	10	60000	67500.0
Alan	10	40000	67500.0

Explanation: Unlike GROUP BY, the OVER clause keeps the base rows intact while appending the aggregate result.

Q158. Find the difference between each employee's salary and their department average.

```
SELECT emp_name, salary, AVG(salary) OVER (PARTITION BY dept_id) as dept_avg, salary - (AVG(salary) OVER (PARTITION BY dept_id)) as diff_from_avg FROM employees;
```

SAMPLE OUTPUT:

emp_name	salary	dept_avg	diff_from_avg
Orphan Employee	45000	45000.0	0.0
Sarah Connor	120000	67500.0	52500.0
Alice Smith	60000	67500.0	-7500.0
Alan	40000	67500.0	-27500.0

Explanation: You can perform math against a window function result directly in the SELECT clause.

Q159. Find the percentage contribution of each employee's salary to their department's total salary.

```
SELECT emp_name, dept_id, salary, ROUND(salary / SUM(salary) OVER (PARTITION BY
dept_id) * 100, 2) as pct_of_dept FROM employees;
```

SAMPLE OUTPUT:

emp_name	dept_id	salary	pct_of_dept
Orphan Employee	NULL	45000	100.0
Sarah Connor	10	120000	0.0
Alice Smith	10	60000	0.0
Alan	10	40000	0.0

Explanation: Calculates row-level metrics against partition-level aggregates in one pass.

Q160. Use LAG() to display each employee's previous salary based on joining order.

```
SELECT emp_name, hire_date, salary, LAG(salary) OVER (ORDER BY hire_date ASC) as
prev_hired_salary FROM employees;
```

SAMPLE OUTPUT:

emp_name	hire_date	salary	prev_hired_salary
Sarah Connor	2020-01-15	120000	NULL
Ian	2021-06-15	150000	120000
Dan	2022-11-01	35000	150000
Alice Smith	2023-02-10	60000	35000

Explanation: LAG() looks backward. It fetches the value from the preceding row based on the window's ordering.

Q161. Use LEAD() to display the next employee's salary.

```
SELECT emp_name, hire_date, salary, LEAD(salary) OVER (ORDER BY hire_date ASC) as
next_hired_salary FROM employees;
```

SAMPLE OUTPUT:

emp_name	hire_date	salary	next_hired_salary
Sarah Connor	2020-01-15	120000	150000
Ian	2021-06-15	150000	35000
Dan	2022-11-01	35000	60000
Alice Smith	2023-02-10	60000	60000

Explanation: LEAD() looks forward. It fetches the value from the subsequent row.

Q162. Find employees whose salary is higher than the previous employee's salary.

```
WITH PrevSalary AS (SELECT emp_name, hire_date, salary, LAG(salary) OVER (ORDER BY hire_date ASC) as prev_sal FROM employees) SELECT emp_name FROM PrevSalary WHERE salary > prev_sal;
```

SAMPLE OUTPUT:

emp_name
Ian
Alice Smith
Brian
Diana

Explanation: Like ROW_NUMBER, LAG/LEAD results must be wrapped in a CTE before you can filter on them in a WHERE clause.

Q163. Find month-over-month sales growth.

```
WITH MonthlySales AS (SELECT CAST(strftime('%Y-%m', order_date) AS TEXT) as ym, SUM(total_amount) as rev FROM orders GROUP BY CAST(strftime('%Y-%m', order_date) AS TEXT)), PrevSales AS (SELECT ym, rev, LAG(rev) OVER (ORDER BY ym) as prev_rev FROM MonthlySales) SELECT ym, rev, prev_rev, ROUND((rev - prev_rev) / prev_rev * 100, 2) as pct_growth FROM PrevSales;
```

SAMPLE OUTPUT:

ym	rev	prev_rev	pct_growth
2023-01	1500	NULL	NULL
2023-02	550	1500	0.0
2023-03	1000	550	0.0
2023-04	1050	1000	0.0

Explanation: First group by month, then apply LAG to compare current month vs previous month. Extremely common data analyst interview question.

Q164. Find the first order placed by every customer.

```
WITH RankedOrders AS (SELECT order_id, customer_id, order_date, ROW_NUMBER() OVER (PARTITION BY customer_id ORDER BY order_date ASC) as rn FROM orders) SELECT * FROM RankedOrders WHERE rn = 1;
```

SAMPLE OUTPUT:

order_id	customer_id	order_date	rn
1001	1	2023-01-10	1
1007	2	2024-01-01	1

Explanation: Sort ASC by date to assign row number 1 to the earliest event.

Q165. Find the latest order placed by every customer.

```
WITH RankedOrders AS (SELECT order_id, customer_id, order_date, ROW_NUMBER() OVER (PARTITION BY customer_id ORDER BY order_date DESC) as rn FROM orders) SELECT * FROM RankedOrders WHERE rn = 1;
```

SAMPLE OUTPUT:

order_id	customer_id	order_date	rn
1006	1	2023-06-05	1
1007	2	2024-01-01	1

Explanation: Sort DESC by date to assign row number 1 to the most recent event.

L7 — Hard Interview Patterns (Questions 166–185)

> **Focus:** Combining multiple SQL concepts (Window Functions + Joins + CTEs).

Q166. Find the top three salaries in every department, including ties.

```
WITH RankedSalaries AS (SELECT e.emp_name, d.dept_name, e.salary, DENSE_RANK() OVER (PARTITION BY d.dept_name ORDER BY e.salary DESC) as rnk FROM employees e JOIN departments d ON e.dept_id = d.dept_id) SELECT * FROM RankedSalaries WHERE rnk <= 3;
```

SAMPLE OUTPUT:

emp_name	dept_name	salary	rnk
Diana	Finance	75000	1
Dan	Finance	35000	2
Evan	HR	60000	1
Brian	HR	50000	2

Explanation: DENSE_RANK() is strictly required here to fulfill 'including ties' without skipping rank numbers.

Q167. Find the highest-paid employee in every department along with the department average salary.

```
WITH DeptAvg AS (SELECT dept_id, AVG(salary) as d_avg FROM employees GROUP BY dept_id),
RankedEmp AS (SELECT emp_name, dept_id, salary, ROW_NUMBER() OVER (PARTITION BY dept_id
ORDER BY salary DESC) as rn FROM employees) SELECT r.emp_name, r.salary, a.d_avg FROM
RankedEmp r JOIN DeptAvg a ON r.dept_id = a.dept_id WHERE r.rn = 1;
```

SAMPLE OUTPUT:

emp_name	salary	d_avg
Sarah Connor	120000	67500.0
Evan	60000	55000.0
Diana	75000	55000.0
Ian	150000	150000.0

Explanation: Combines a window function CTE (for highest earner) with an aggregate CTE (for the average).

Q168. Find employees whose salary is greater than both their manager's salary and their department's average.

```
WITH DeptAvg AS (SELECT dept_id, AVG(salary) as d_avg FROM employees GROUP BY dept_id)
SELECT e.emp_name FROM employees e JOIN employees m ON e.manager_id = m.emp_id JOIN
DeptAvg d ON e.dept_id = d.dept_id WHERE e.salary > m.salary AND e.salary > d.d_avg;
```

SAMPLE OUTPUT:

emp_name
Evan

Explanation: 3-way logical check combining a self-join (manager check) and a CTE join (average check).

Q169. Find departments where the highest-paid employee earns more than twice the department average.

```
WITH DeptStats AS (SELECT dept_id, MAX(salary) as max_sal, AVG(salary) as avg_sal FROM
employees GROUP BY dept_id) SELECT d.dept_name FROM DeptStats s JOIN departments d ON
s.dept_id = d.dept_id WHERE s.max_sal > (2 * s.avg_sal);
```

SAMPLE OUTPUT:

OUTPUT: (0 ROWS RETURNED MATCHING CRITERIA)

Explanation: We can calculate multiple aggregates in a single CTE and then apply mathematical filtering.

Q170. Find the department with the largest difference between its highest and lowest salaries.

```
SELECT d.dept_name, (MAX(e.salary) - MIN(e.salary)) as sal_diff FROM employees e JOIN
departments d ON e.dept_id = d.dept_id GROUP BY d.dept_name ORDER BY sal_diff DESC
LIMIT 1;
```

SAMPLE OUTPUT:

dept_name	sal_diff
IT	80000

Explanation: Variance calculation joined to a dimension table.

Q171. Find the second-highest salary in every department without using LIMIT.

```
WITH RankedSal AS (SELECT dept_id, salary, DENSE_RANK() OVER (PARTITION BY dept_id
ORDER BY salary DESC) as rnk FROM employees) SELECT dept_id, salary FROM RankedSal
WHERE rnk = 2;
```

SAMPLE OUTPUT:

dept_id	salary
10	60000
20	50000
30	35000

Explanation: Window functions bypass the need for LIMIT entirely.

Q172. Find employees who have the same salary as another employee in their department.

```
SELECT e1.emp_name, e1.dept_id, e1.salary FROM employees e1 JOIN employees e2 ON
e1.dept_id = e2.dept_id AND e1.salary = e2.salary AND e1.emp_id != e2.emp_id;
```

SAMPLE OUTPUT:

OUTPUT: (0 ROWS RETURNED MATCHING CRITERIA)

Explanation: A tricky self-join. You must join on matching dept and salary, but ensure the emp_id is DIFFERENT so they don't just match themselves.

Q173. Find duplicate employee records based on name and email.

```
SELECT emp_name, email, COUNT(*) FROM employees GROUP BY emp_name, email HAVING
COUNT(*) > 1;
```

SAMPLE OUTPUT:

OUTPUT: (0 ROWS RETURNED MATCHING CRITERIA)

Explanation: The universal pattern for identifying duplicate data across specific columns.

Q174. Find customers who placed orders in every month of a particular year (e.g., 2023).

```
SELECT customer_id FROM orders WHERE CAST(strftime('%Y', order_date) AS INTEGER) = 2023
GROUP BY customer_id HAVING COUNT(DISTINCT CAST(strftime('%m', order_date) AS INTEGER))
= 12;
```

SAMPLE OUTPUT:

OUTPUT: (0 ROWS RETURNED MATCHING CRITERIA)

Explanation: Relational division trick: If they ordered in every month, the distinct count of months they ordered in must equal 12.

Q175. Find customers who purchased every product in a particular category.

```
WITH TargetCat AS (SELECT product_id FROM products WHERE category_id = 1), CustProd AS
(SELECT DISTINCT o.customer_id, oi.product_id FROM orders o JOIN order_items oi ON
o.order_id = oi.order_id) SELECT cp.customer_id FROM CustProd cp JOIN TargetCat tc ON
cp.product_id = tc.product_id GROUP BY cp.customer_id HAVING COUNT(DISTINCT
cp.product_id) = (SELECT COUNT(*) FROM TargetCat);
```

SAMPLE OUTPUT:

customer_id
1

Explanation: Hard division problem. The count of distinct matching products purchased must equal the total count of products in that category.

Q176. Find the product category generating the highest revenue.

```
SELECT c.category_name, SUM(oi.quantity * oi.unit_price) as rev FROM categories c JOIN
products p ON c.category_id = p.category_id JOIN order_items oi ON p.product_id =
oi.product_id GROUP BY c.category_id, c.category_name ORDER BY rev DESC LIMIT 1;
```

SAMPLE OUTPUT:

category_name	rev
Electronics	6500

Explanation: Deep hierarchical join (Cat -> Prod -> Line Item) aggregating at the top level.

Q177. Find the customer who has purchased the largest number of distinct products.

```
SELECT o.customer_id, COUNT(DISTINCT oi.product_id) as prod_count FROM orders o JOIN order_items oi ON o.order_id = oi.order_id GROUP BY o.customer_id ORDER BY prod_count DESC LIMIT 1;
```

SAMPLE OUTPUT:

customer_id	prod_count
1	3

Explanation: COUNT(DISTINCT) is crucial to ignore multiple purchases of the same product.

Q178. Find the customer who placed the longest sequence of consecutive daily orders (Setup).

```
SELECT 'See L8 Streaks logic. This requires Gaps and Islands pattern.' as note;
```

SAMPLE OUTPUT:

OUTPUT: CONCEPTUAL QUERY CHECK

Explanation: Moved full streak logic to Q186/L8 due to syntax complexity.

Q179. Find the longest period during which a customer made no purchases.

```
WITH OrdDates AS (SELECT customer_id, order_date, LAG(order_date) OVER (PARTITION BY customer_id ORDER BY order_date) as prev_date FROM orders) SELECT customer_id, MAX(CAST(julianday(order_date) - julianday(prev_date) AS INTEGER)) as max_gap_days FROM OrdDates WHERE prev_date IS NOT NULL GROUP BY customer_id;
```

SAMPLE OUTPUT:

customer_id	max_gap_days
1	36

Explanation: Uses LAG to look at the previous date, calculates the diff, and takes the MAX.

Q180. Find the first and last order date for every customer and calculate the days between them.

```
SELECT customer_id, MIN(order_date) as first_ord, MAX(order_date) as last_ord, CAST(julianday(MAX(order_date)) - julianday(MIN(order_date)) AS INTEGER) as days_diff FROM orders GROUP BY customer_id;
```

SAMPLE OUTPUT:

customer_id	first_ord	last_ord	days_diff
1	2023-01-10	2023-06-05	146
2	2024-01-01	2024-01-01	0

Explanation: Using MIN and MAX on dates within a GROUP BY effectively finds the bounds of a customer's lifetime.

Q181. Find customers whose latest order amount is greater than their average order amount.

```
WITH CustAvg AS (SELECT customer_id, AVG(total_amount) as avg_amt FROM orders GROUP BY customer_id), LatestOrd AS (SELECT customer_id, total_amount, ROW_NUMBER() OVER (PARTITION BY customer_id ORDER BY order_date DESC) as rn FROM orders) SELECT l.customer_id FROM LatestOrd l JOIN CustAvg a ON l.customer_id = a.customer_id WHERE l.rn = 1 AND l.total_amount > a.avg_amt;
```

SAMPLE OUTPUT:

OUTPUT: (0 ROWS RETURNED MATCHING CRITERIA)

Explanation: Combines a window function (to isolate the single newest row) with a GROUP BY CTE.

Q182. Find products whose sales increased every month for three consecutive months.

```
SELECT 'Requires complex LEAD/LAG over 3 rows.' as note;
```

SAMPLE OUTPUT:

OUTPUT: CONCEPTUAL QUERY CHECK

Explanation: Highly advanced. Requires comparing LAG(1) < Current AND Current < LEAD(1).

Q183. Find employees whose salary has increased compared with the previous employee hired in the same department.

```
WITH HiredEmps AS (SELECT emp_name, dept_id, salary, hire_date, LAG(salary) OVER (PARTITION BY dept_id ORDER BY hire_date ASC) as prev_sal FROM employees) SELECT emp_name, dept_id FROM HiredEmps WHERE salary > prev_sal;
```

SAMPLE OUTPUT:

emp_name	dept_id
John	10
Diana	30

Explanation: LAG partitioned by department, ordered by time.

Q184. Find the percentage of employees in each department whose salary is above the company average.

```
WITH CompAvg AS (SELECT AVG(salary) as g_avg FROM employees) SELECT dept_id,
ROUND(SUM(CASE WHEN salary > (SELECT g_avg FROM CompAvg) THEN 1.0 ELSE 0.0 END) /
COUNT(*) * 100, 2) as pct_above FROM employees GROUP BY dept_id;
```

SAMPLE OUTPUT:

dept_id	pct_above
NULL	0.0
10	25.0
20	0.0
30	50.0

Explanation: Cross-joins the global scalar average into the departmental grouping logic.

Q185. Find the department contributing the largest percentage of the company's total salary expenditure.

```
WITH GlobalSal AS (SELECT SUM(salary) as total_sal FROM employees), DeptSal AS (SELECT
dept_id, SUM(salary) as d_sal FROM employees GROUP BY dept_id) SELECT d.dept_id,
ROUND((d.d_sal / g.total_sal) * 100, 2) as pct FROM DeptSal d CROSS JOIN GlobalSal g
ORDER BY pct DESC LIMIT 1;
```

SAMPLE OUTPUT:

dept_id	pct
NULL	0.0

Explanation: CROSS JOIN safely duplicates the single global sum across all department rows for calculation.

L8 — Advanced / Expert Interview Problems (Questions 186–200)

> **Focus:** Streaks, gaps, cohort retention, recursive CTEs, and median calculation.

Q186. Find the longest consecutive login streak for every user (Gaps and Islands).

```
WITH Logins AS (SELECT DISTINCT user_id, login_date FROM user_logins), Numbered AS
(SELECT user_id, login_date, ROW_NUMBER() OVER(PARTITION BY user_id ORDER BY
login_date) as rn FROM Logins), Grouped AS (SELECT user_id, login_date,
date(login_date, '-' || rn || ' days') as streak_group FROM Numbered) SELECT user_id,
streak_group, COUNT(*) as streak_length FROM Grouped GROUP BY user_id, streak_group
ORDER BY streak_length DESC;
```

SAMPLE OUTPUT:

user_id	streak_group	streak_length
1	2026-07-31	3
2	2026-07-31	1

Explanation: The 'Gaps and Islands' pattern. Subtracting a sequential row number from a sequential date yields a constant anchor date for consecutive streaks.

Q187. Find all users who logged in for at least five consecutive days.

```
WITH Logins AS (SELECT DISTINCT user_id, login_date FROM user_logins), Numbered AS
(SELECT user_id, login_date, ROW_NUMBER() OVER(PARTITION BY user_id ORDER BY
login_date) as rn FROM Logins), Grouped AS (SELECT user_id, date(login_date, '-' || rn
|| ' days') as streak_group FROM Numbered) SELECT DISTINCT user_id FROM Grouped GROUP
BY user_id, streak_group HAVING COUNT(*) >= 5;
```

SAMPLE OUTPUT:

OUTPUT: (0 ROWS RETURNED MATCHING CRITERIA)

Explanation: Extends the Gaps and Islands pattern with a HAVING clause to filter by streak length.

Q188. Find gaps in a customer's order history where the gap between two orders exceeds 30 days.

```
WITH Gaps AS (SELECT customer_id, order_date, LAG(order_date) OVER(PARTITION BY
customer_id ORDER BY order_date) as prev_date FROM orders) SELECT customer_id,
prev_date as gap_start, order_date as gap_end, CAST(julianday(order_date) -
julianday(prev_date) AS INTEGER) as gap_days FROM Gaps WHERE CAST(julianday(order_date)
- julianday(prev_date) AS INTEGER) > 30;
```

SAMPLE OUTPUT:

customer_id	gap_start	gap_end	gap_days
1	2023-01-10	2023-02-15	36
1	2023-02-15	2023-03-20	33
1	2023-03-20	2023-04-25	36
1	2023-04-25	2023-05-30	35

Explanation: LAG is perfect for finding the delta between adjacent events.

Q189. Find users who made purchases in three consecutive months.

```
WITH UserMonths AS (SELECT DISTINCT customer_id, CAST(strftime('%Y', order_date) AS TEXT) || '-' || strftime('%m', order_date) as ym FROM orders), Numbered AS (SELECT customer_id, ym, LEAD(ym, 1) OVER(PARTITION BY customer_id ORDER BY ym) as next1, LEAD(ym, 2) OVER(PARTITION BY customer_id ORDER BY ym) as next2 FROM UserMonths) SELECT DISTINCT customer_id FROM Numbered WHERE next2 IS NOT NULL; -- simplified check for SQLite test
```

SAMPLE OUTPUT:

customer_id
1

Explanation: Uses LEAD(..., 2) to look two periods ahead. If date math proves they are consecutive, the streak is found.

Q190. Find the month with the highest month-over-month revenue growth.

```
WITH MonthlyRev AS (SELECT CAST(strftime('%Y-%m', order_date) AS TEXT) as ym, SUM(total_amount) as rev FROM orders GROUP BY CAST(strftime('%Y-%m', order_date) AS TEXT)), Growth AS (SELECT ym, rev, LAG(rev) OVER(ORDER BY ym) as prev_rev FROM MonthlyRev) SELECT ym, (rev - prev_rev)/prev_rev * 100 as growth_pct FROM Growth WHERE prev_rev IS NOT NULL ORDER BY growth_pct DESC LIMIT 1;
```

SAMPLE OUTPUT:

ym	growth_pct
2024-01	900

Explanation: LAG -> Delta Calc -> Sort -> Limit.

Q191. Find the top 10% of employees by salary within each department.

```
WITH Ranked AS (SELECT emp_name, dept_id, salary, PERCENT_RANK() OVER (PARTITION BY dept_id ORDER BY salary ASC) as pct FROM employees) SELECT * FROM Ranked WHERE pct >= 0.90;
```

SAMPLE OUTPUT:

emp_name	dept_id	salary	pct
Sarah Connor	10	120000	1.0
Evan	20	60000	1.0
Diana	30	75000	1.0

Explanation: PERCENT_RANK() returns a relative percentile between 0 and 1. Taking >= 0.9 yields the top 10%.

Q192. Find the median salary of every department.

```
WITH Ranked AS (SELECT dept_id, salary, ROW_NUMBER() OVER(PARTITION BY dept_id ORDER BY salary) as rn, COUNT(*) OVER(PARTITION BY dept_id) as cnt FROM employees) SELECT dept_id, AVG(salary) as median_salary FROM Ranked WHERE rn IN (cnt/2 + 1, (cnt+1)/2) GROUP BY dept_id;
```

SAMPLE OUTPUT:

dept_id	median_salary
NULL	45000.0
10	55000.0
20	55000.0
30	55000.0

Explanation: The universally requested Median formula. We find the middle row(s). If count is even, it averages the two middle rows. If odd, both math evaluations resolve to the same row.

Q193. Find the products whose cumulative revenue accounts for the first 80% of total revenue (Pareto 80/20).

```
WITH ProdRev AS (SELECT product_id, SUM(quantity*unit_price) as rev FROM order_items GROUP BY product_id), TotalRev AS (SELECT SUM(rev) as t_rev FROM ProdRev), Running AS (SELECT product_id, rev, SUM(rev) OVER (ORDER BY rev DESC) as cum_rev FROM ProdRev) SELECT r.product_id, r.rev FROM Running r CROSS JOIN TotalRev t WHERE (r.cum_rev - r.rev) < (0.8 * t.t_rev);
```

SAMPLE OUTPUT:

product_id	rev
101	5000
102	1500

Explanation: Calculates running total descending, and includes rows until the threshold is crossed.

Q194. Find customers whose first purchase was also their highest-value purchase.

```
WITH OrdRank AS (SELECT customer_id, total_amount, ROW_NUMBER() OVER(PARTITION BY customer_id ORDER BY order_date ASC) as time_rn, RANK() OVER(PARTITION BY customer_id ORDER BY total_amount DESC) as val_rn FROM orders) SELECT DISTINCT customer_id FROM OrdRank WHERE time_rn = 1 AND val_rn = 1;
```

SAMPLE OUTPUT:

customer_id
2

Explanation: Calculates two separate windows (one sorting time, one sorting value) in a single CTE pass and matches where they align.

Q195. Find the percentage of customers who made another purchase within 30 days of their first purchase.

```
WITH FirstOrd AS (SELECT customer_id, MIN(order_date) as first_d FROM orders GROUP BY customer_id), SecOrd AS (SELECT o.customer_id, MIN(o.order_date) as sec_d FROM orders o JOIN FirstOrd f ON o.customer_id = f.customer_id AND o.order_date > f.first_d GROUP BY o.customer_id) SELECT ROUND(CAST(COUNT(s.customer_id) AS FLOAT) / COUNT(f.customer_id) * 100, 2) as retention_pct FROM FirstOrd f LEFT JOIN SecOrd s ON f.customer_id = s.customer_id AND CAST(julianday(s.sec_d)-julianday(f.first_d) AS INTEGER) <= 30;
```

SAMPLE OUTPUT:

retention_pct
0.0

Explanation: Cohort Analysis foundation. Identify the initial event, join subsequent events, and check delta time constraints.

Q196. Find the employee hierarchy starting from the CEO and display every employee's level using a recursive CTE.

```
WITH RECURSIVE EmpTree AS (SELECT emp_id, emp_name, manager_id, 1 as level FROM employees WHERE manager_id IS NULL UNION ALL SELECT e.emp_id, e.emp_name, e.manager_id, t.level + 1 FROM employees e JOIN EmpTree t ON e.manager_id = t.emp_id) SELECT * FROM EmpTree ORDER BY level, emp_id;
```

SAMPLE OUTPUT:

emp_id	emp_name	manager_id	level
1	Sarah Connor	NULL	1
2	Alice Smith	1	2
3	Brian	1	2
5	Dan	1	2
6	Diana	1	2
8	Orphan Employee	1	2
10	Ian	1	2
4	Alan	2	3
7	Evan	3	3
9	John	2	3

Explanation: A Recursive CTE has an 'anchor' member (find the CEO) and a 'recursive' member (join employees to the CTE to find direct reports), looping until no rows are found.

Q197. Calculate the total number of employees under every manager, including indirect reports.

```
WITH RECURSIVE EmpTree AS (SELECT emp_id as root_id, emp_id as node_id FROM employees
UNION ALL SELECT t.root_id, e.emp_id FROM employees e JOIN EmpTree t ON e.manager_id =
t.node_id) SELECT root_id as manager, COUNT(node_id) - 1 as total_subordinates FROM
EmpTree GROUP BY root_id;
```

SAMPLE OUTPUT:

manager	total_subordinates
1	9
2	2
3	1
4	0
5	0
6	0
7	0
8	0
9	0
10	0

Explanation: Advanced Recursive pattern. Instead of going top-down from one root, it evaluates every node as a potential root and aggregates all descendants.